Base de datos con PHP

PHP, uno de los lenguajes de programación más utilizados en el desarrollo web, nos brinda una amplia gama de herramientas y funcionalidades que permiten interactuar con bases de datos de manera sencilla y efectiva, ofreciendo un entorno flexible y robusto para gestionar la información.

Métodos comunes para conectarse a una Base de datos:
Hay varios métodos populares para conectarse a una base de datos. A continuación los más comunes

1. **Extensión MySQLi (MySQL Improved):**
   MySQLi es una extensión específica para trabajar con bases de datos MySQL. Proporciona una interfaz *(*1) orientada a objetos* y *una interfaz procedural* . Algunos métodos importantes son **mysqli_connect()** para establecer la conexión, **mysqli_query()** para ejecutar consultas y **mysqli_fetch_assoc()** para obtener filas de resultados. También ofrece soporte para consultas preparadas y transacciones.

2. **PDO (PHP Data Objects):**
   PDO es una capa de abstracción que proporciona una interfaz orientada a objetos para interactuar con diferentes bases de datos. Puedes utilizar el constructor PDO para establecer la conexión, query() para ejecutar consultas y fetchAll() para obtener resultados. PDO es compatible con varios sistemas de bases de datos, lo que te brinda portabilidad y seguridad.

3. **Extensión MySQL (Deprecated):**
   La extensión MySQL original, aunque ahora se considera obsoleta, todavía se utiliza en algunos sistemas heredados. Proporciona una interfaz procedural para trabajar con bases de datos MySQL. Algunos métodos comunes son mysql_connect() para conectarse a la base de datos, mysql_query() para ejecutar consultas y mysql_fetch_assoc() para obtener resultados.

(*1) MySQLi (MySQL Improved): La interfaz orientada a objetos de MySQLi (MySQL Improved) en PHP te permite interactuar con la base de datos utilizando objetos y métodos específicos. Proporciona una forma más estructurada y orientada a objetos de trabajar con la base de datos.

En la interfaz orientada a objetos de MySQLi, se utilizan clases y métodos para realizar operaciones relacionadas con la base de datos. A continuación, un ejemplo de cómo utilizar la interfaz orientada a objetos para establecer una conexión, ejecutar una consulta y obtener resultados:

```
// Establecer la conexión a la base de datos
$mysqli = new mysqli("localhost", "usuario", "contraseña", "nombre_base_de_datos");

// Verificar si hay errores de conexión
if ($mysqli->connect_error) {
    die("Error de conexión: " . $mysqli->connect_error);
}

// Ejecutar una consulta
$query = "SELECT * FROM tabla";
$result = $mysqli->query($query);

// Verificar si hay errores en la consulta
if (!$result) {
    die("Error en la consulta: " . $mysqli->error);
}

// Obtener los resultados de la consulta
while ($row = $result->fetch_assoc()) {
    echo $row["columna1"] . " - " . $row["columna2"] . "<br>";
}

// Cerrar la conexión
$mysqli->close();
```

Una interfaz procedural en el contexto de la extensión MySQLi (MySQL Improved) se refiere a la forma en que puedes interactuar con la base de datos utilizando funciones y llamadas de procedimientos, en lugar de una interfaz orientada a objetos.

Es una forma más tradicional y procedural de trabajar con la base de datos,como establecer una conexión, ejecutar consultas y obtener resultados.

En el enfoque procedural, las operaciones se realizan mediante llamadas a funciones individuales. Por ejemplo, para establecer una conexión a la base de datos utilizando MySQLi en estilo procedural, puedes utilizar la función **mysqli_connect()**, entre otras.

Aquí tienes un ejemplo de cómo utilizar la interfaz procedural de MySQLi para establecer una conexión, ejecutar una consulta y obtener resultados:

```php
// Establecer la conexión a la base de datos
$link = mysqli_connect("localhost", "usuario", "contraseña", "nombre_base_de_datos");

// Verificar si hay errores de conexión
if (!$link) {
    die("Error de conexión: " . mysqli_connect_error());
}

// Ejecutar una consulta
$query = "SELECT * FROM tabla";
$result = mysqli_query($link, $query);

// Verificar si hay errores en la consulta
if (!$result) {
    die("Error en la consulta: " . mysqli_error($link));
}

// Obtener los resultados de la consulta
while ($row = mysqli_fetch_assoc($result)) {
    echo $row["columna1"] . " - " . $row["columna2"] . "<br>";
}

// Cerrar la conexión
mysqli_close($link);
```

En el ejemplo anterior, utilizamos la función **mysqli_connect()** para establecer una conexión con la base de datos. Luego, verificamos si hay errores de conexión utilizando **mysqli_connect_error()**.

Para ejecutar una consulta, utilizamos la función **mysqli_query()** pasando la conexión **$link** y la consulta SQL. Después, verificamos si hay errores en la consulta utilizando **mysqli_error()**.

Para obtener los resultados de la consulta, utilizamos la función **mysqli_fetch_assoc()** en un bucle while para iterar sobre las filas devueltas por la consulta.

Finalmente, cerramos la conexión utilizando **mysqli_close()**.

La interfaz procedural de MySQLi se basa en el uso de funciones específicas para interactuar con la base de datos. Aunque la interfaz orientada a objetos es más comúnmente utilizada y considerada más moderna, la interfaz procedural puede ser más familiar y cómoda para aquellos que están acostumbrados a un enfoque procedural en la programación.

PDO (PHP Data Objects)

PDO es una extensión en PHP que proporciona una interfaz consistente y orientada a objetos para acceder y manipular bases de datos. Ofrece una capa de abstracción que permite a los desarrolladores trabajar con diferentes motores de bases de datos, como MySQL, PostgreSQL, SQLite y más, utilizando una sintaxis común y un conjunto de métodos coherentes.

Una capa de abstracción, en el contexto de las bases de datos, se refiere a una interfaz que oculta los detalles específicos de un motor de base de datos subyacente y proporciona un conjunto común de métodos y funciones para interactuar con diferentes motores de bases de datos. En esencia, actúa como una capa intermedia entre la aplicación y la base de datos, simplificando y unificando el acceso a los datos.

La capa de abstracción permite a los desarrolladores trabajar con diferentes motores de bases de datos utilizando una sintaxis y un conjunto de métodos consistentes, independientemente de las diferencias en la sintaxis SQL o en la implementación específica de cada motor. Esto facilita la portabilidad de la aplicación, ya que puede cambiar de un motor de base de datos a otro sin tener que realizar cambios significativos en el código de la aplicación.

Además, la capa de abstracción puede proporcionar características adicionales, como consultas preparadas, manejo de transacciones y gestión de errores, que pueden no estar presentes en todos los motores de bases de datos de forma nativa. Esto permite que la funcionalidad se mantenga consistente en diferentes entornos de bases de datos.

En el caso de PDO, es una capa de abstracción de base de datos en PHP que proporciona una interfaz consistente y orientada a objetos para trabajar con diferentes motores de bases de datos. Permite a los desarrolladores escribir código de manera independiente del motor de base de datos subyacente, ya que se encarga de adaptar las consultas y operaciones a la sintaxis y características específicas de cada motor.

Podemos enunciar sus principales características clave de la siguiente manera:

- **Conexión a la base de datos**: PDO permite establecer conexiones a diferentes motores de bases de datos utilizando una sintaxis uniforme. Puedes especificar el tipo de base de datos, el host, el nombre de usuario, la contraseña y otras opciones relevantes.

- **Consultas preparadas**: PDO proporciona soporte para consultas preparadas, lo que permite separar la lógica de la consulta del contenido de los datos. Esto ayuda a prevenir ataques de inyección SQL y mejora la seguridad de tu aplicación.

- **Soporte para transacciones**: PDO admite transacciones, lo que te permite agrupar varias consultas en una unidad lógica y asegurarte de que se completen todas correctamente o se deshagan todas si ocurre algún error.

- **Diferentes métodos de obtención de resultados**: PDO ofrece varias opciones para obtener los resultados de una consulta, como **fetch**(), **fetchAll**(), **fetchColumn**(), entre otros. Estos métodos te permiten recuperar los datos de diferentes maneras, como un solo resultado, múltiples filas o una única columna.

- **Gestión de errores**: PDO proporciona manejo de errores a través de excepciones, lo que facilita la detección y el manejo de errores relacionados con la base de datos de manera más consistente y estructurada.

- **Soporte para controladores de base de datos**: PDO utiliza controladores específicos para cada motor de base de datos. Esto significa que puedes cambiar de un motor a otro simplemente modificando la cadena de conexión y no tienes que modificar el código de tu aplicación.

- **Características adicionales**: PDO también ofrece características adicionales como vinculación de parámetros, gestión de blobs (datos binarios grandes), recuperación de metadatos de la base de datos, entre otros.

Ejemplo básico de cómo utilizar PDO para establecer una conexión a una base de datos MySQL y ejecutar una consulta:

```
// Establecer la conexión a la base de datos
$dsn = "mysql:host=localhost;dbname=nombre_base_de_datos;charset=utf8";
$usuario = "nombre_usuario";
$contraseña = "contraseña";
$opciones = array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION);
$pdo = new PDO($dsn, $usuario, $contraseña, $opciones);

// Ejecutar una consulta
$consulta = "SELECT * FROM tabla";
$resultado = $pdo->query($consulta);

// Obtener los resultados de la consulta
while ($fila = $resultado->fetch(PDO::FETCH_ASSOC)) {
    // Acceder a los datos de la fila
    echo $fila['columna1'] . " - " . $fila['columna2'] . "<br>";
}

// Cerrar la conexión
$pdo = null;
```

Ejemplo de cómo utilizar consultas preparadas en PDO

```
// Establecer la conexión a la base de datos
$dsn = "mysql:host=localhost;dbname=nombre_base_de_datos;charset=utf8";
$usuario = "nombre_usuario";
$contraseña = "contraseña";
$opciones = array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION);
$pdo = new PDO($dsn, $usuario, $contraseña, $opciones);

// Ejemplo de consulta preparada
$nombre = "John";
$apellido = "Doe";

// Preparar la consulta
$consulta = $pdo->prepare("SELECT * FROM usuarios WHERE nombre = :nombre AND apellido = :apellido");

// Vincular los parámetros
$consulta->bindParam(":nombre", $nombre);
$consulta->bindParam(":apellido", $apellido);

// Ejecutar la consulta
$consulta->execute();

// Obtener los resultados
$resultados = $consulta->fetchAll(PDO::FETCH_ASSOC);

// Recorrer los resultados
foreach ($resultados as $fila) {
    echo $fila['nombre'] . " " . $fila['apellido'] . "<br>";
}

// Cerrar la conexión
$pdo = null;
```

En el ejemplo anterior, primero establecemos la conexión a la base de datos utilizando PDO. Luego, preparamos una consulta utilizando la función **prepare**(), donde utilizamos marcadores de posición :**nombre** y :**apellido** en lugar de los valores reales.

Después, utilizamos la función **bindParam**() para vincular los parámetros de la consulta a las variables $**nombre** y $**apellido**. Esto permite que los valores se pasen de forma segura a la consulta *sin estar expuestos a ataques de inyección SQL.*

Luego, ejecutamos la consulta con **execute**(), que ejecuta la consulta preparada con los valores proporcionados. Los resultados se obtienen utilizando **fetchAll**() y se almacenan en el arreglo $**resultados**.

Finalmente, recorremos los resultados y los mostramos en el bucle foreach.

El uso de consultas preparadas en PDO ayuda a prevenir ataques de inyección SQL y mejora la seguridad de tu aplicación al separar la lógica de la consulta de los datos proporcionados por el usuario. Además, también puede mejorar el rendimiento de las consultas, ya que la consulta preparada se compila una vez y luego se pueden ejecutar múltiples veces con diferentes valores sin necesidad de volver a compilarla.

¿Es necesario cerrar la conexión?
A partir de PHP 5.3.6, el cierre explícito de la conexión en PDO no es necesario. PHP se encarga automáticamente de cerrar la conexión cuando el objeto PDO es destruido, es decir, cuando ya no hay referencias a él o cuando se ejecuta la instrucción unset($pdo)

Ejemplo de cómo utilizar transacciones en PDO:

```
// Establecer la conexión a la base de datos
$dsn = "mysql:host=localhost;dbname=nombre_base_de_datos;charset=utf8";
$usuario = "nombre_usuario";
$contraseña = "contraseña";
$opciones = array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION);
$pdo = new PDO($dsn, $usuario, $contraseña, $opciones);

try {
    // Iniciar la transacción
    $pdo->beginTransaction();

    // Ejecutar consultas dentro de la transacción
    $pdo->exec("UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1");
    $pdo->exec("UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2");

    // Confirmar la transacción
    $pdo->commit();

    echo "Transacción completada con éxito.";
} catch (PDOException $e) {
    // En caso de error, deshacer la transacción
    $pdo->rollBack();
    echo "Error en la transacción: " . $e->getMessage();
}

// Cerrar la conexión
$pdo = null;
```

En el ejemplo anterior, primero se establece la conexión a la base de datos utilizando PDO. Luego, se inicia la transacción utilizando el método **beginTransaction**(). Todas las consultas posteriores se ejecutarán dentro de esta transacción.

En este caso, se realizan dos consultas de actualización en la tabla "cuentas", que restan 100 unidades de saldo de la cuenta 1 y suman 100 unidades de saldo a la cuenta 2.

Después de ejecutar las consultas, se confirma la transacción utilizando el método **commit**(). Esto indica que todas las consultas se han realizado con éxito y los cambios deben ser guardados permanentemente en la base de datos.

Si ocurre un error durante la transacción, se captura la excepción **PDOException** y se deshace la transacción utilizando el método **rollBack**(). Esto revierte todos los cambios realizados dentro de la transacción y devuelve la base de datos a su estado anterior.

Finalmente, se cierra la conexión utilizando $pdo = null;.

El uso de transacciones en PDO te permite agrupar un conjunto de consultas en una unidad lógica y garantizar que todas se completen correctamente o se deshagan en caso de error. Esto es especialmente útil cuando se realizan operaciones críticas que deben ser atómicas y consistentes en la base de datos.

Ejemplos de los diferentes métodos de obtención de resultados en PDO:

1-**Método fetch():** Recupera la siguiente fila de resultados como un arreglo asociativo, un objeto o ambos, dependiendo del modo de recuperación especificado. Puedes llamar a este método repetidamente para obtener las filas restantes.

```
// Ejemplo de fetch() con modo de recuperación asociativo
$consulta = $pdo->query("SELECT * FROM usuarios");
while ($fila = $consulta->fetch(PDO::FETCH_ASSOC)) {
    echo $fila['nombre'] . " " . $fila['apellido'] . "<br>";
}
```

2-**Método fetchAll():** Recupera todas las filas de resultados como un arreglo multidimensional, donde cada fila es un arreglo asociativo, un objeto o ambos, dependiendo del modo de recuperación especificado.

```
// Ejemplo de fetchAll() con modo de recuperación objeto
$consulta = $pdo->query("SELECT * FROM usuarios");
$resultados = $consulta->fetchAll(PDO::FETCH_OBJ);
foreach ($resultados as $fila) {
    echo $fila->nombre . " " . $fila->apellido . "<br>";
}
```

3-**Método fetchColumn()**: Recupera el valor de una columna de la siguiente fila de resultados.

```
// Ejemplo de fetchColumn() para obtener un solo valor
$consulta = $pdo->query("SELECT COUNT(*) FROM usuarios");
$total = $consulta->fetchColumn();
echo "Total de usuarios: " . $total;
```

4-**Método fetchObject()**: Recupera la siguiente fila de resultados como un objeto de una clase personalizada o de la clase predefinida stdClass.

```
// Ejemplo de fetchObject() con clase personalizada
class Usuario {
    public $nombre;
    public $apellido;
}

$consulta = $pdo->query("SELECT * FROM usuarios");
while ($fila = $consulta->fetchObject('Usuario')) {
    echo $fila->nombre . " " . $fila->apellido . "<br>";
}
```

Ejemplo de cómo gestionar los errores en PDO utilizando excepciones:

```
try {
    // Establecer la conexión a la base de datos
    $dsn = "mysql:host=localhost;dbname=nombre_base_de_datos;charset=utf8";
    $usuario = "nombre_usuario";
    $contraseña = "contraseña";
    $opciones = array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION);
    $pdo = new PDO($dsn, $usuario, $contraseña, $opciones);

    // Ejecutar consultas u operaciones con la base de datos aquí

} catch (PDOException $e) {
    // Capturar la excepción PDOException y mostrar un mensaje de error
    echo "Error en la conexión: " . $e->getMessage();
}
```

En el ejemplo anterior, se utiliza un bloque **try-catch** para capturar cualquier excepción lanzada por PDO. Esto nos permite gestionar y manejar los errores de manera controlada.

Dentro del bloque **try**, se establece la conexión a la base de datos y se ejecutan las consultas u operaciones correspondientes. Si ocurre un error durante la ejecución, se lanzará una excepción de tipo **PDOException**.

En el bloque **catch**, se captura la **excepción PDOException** y se muestra un mensaje de error con el detalle del problema utilizando el método **getMessage**().

El uso de excepciones en PDO nos permite tener un manejo más estructurado y detallado de los errores que pueden ocurrir durante la conexión y las operaciones con la base de datos. Además, podemos personalizar la forma en que se manejan los errores, como registrarlos en un archivo de registro o redirigir a una página de error personalizada.

Algunos Ejemplos de algunas características adicionales que puedes utilizar en PDO:

Obtener el último ID insertado:

```
// Ejemplo de obtener el último ID insertado
$consulta = $pdo->prepare("INSERT INTO usuarios (nombre, apellido) VALUES (:nombre, :apellido)");
$consulta->bindParam(':nombre', $nombre);
$consulta->bindParam(':apellido', $apellido);
$consulta->execute();

$ultimoID = $pdo->lastInsertId();
```

Manejo de transacciones:

```
// Ejemplo de manejo de transacciones
try {
    $pdo->beginTransaction();

    // Realizar operaciones dentro de la transacción

    $pdo->commit();
} catch (PDOException $e) {
    $pdo->rollBack();
    echo "Error en la transacción: " . $e->getMessage();
}
```

Consultas preparadas con múltiples conjuntos de datos:

```
// Ejemplo de consulta preparada con múltiples conjuntos de datos
$consulta = $pdo->prepare("SELECT * FROM usuarios WHERE edad > :edad; SELECT * FROM productos WHERE
precio < :precio");
$consulta->bindParam(':edad', $edad);
$consulta->bindParam(':precio', $precio);
$consulta->execute();

do {
    $resultados = $consulta->fetchAll(PDO::FETCH_ASSOC);
    // Procesar los resultados de cada conjunto de datos
} while ($consulta->nextRowset());
```

Insertar datos preparados:

```
// Datos a insertar
$nombre = 'John Doe';
$email = 'johndoe@example.com';
$contrasena = 'secreto123';

try {
    // Establecer la conexión a la base de datos
    $dsn = "mysql:host=localhost;dbname=nombre_base_de_datos;charset=utf8";
    $usuario = "nombre_usuario";
    $contrasena = "contraseña";
    $opciones = array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION);
    $pdo = new PDO($dsn, $usuario, $contrasena, $opciones);

    // Preparar la consulta de inserción
    $consulta = $pdo->prepare("INSERT INTO usuarios (nombre, email, contrasena) VALUES (:nombre,
:email, :contrasena)");

    // Vincular los parámetros
    $consulta->bindParam(':nombre', $nombre);
    $consulta->bindParam(':email', $email);
    $consulta->bindParam(':contrasena', $contrasena);

    // Ejecutar la consulta
    $consulta->execute();

    echo "Registro insertado con éxito.";
} catch (PDOException $e) {
    echo "Error en la inserción: " . $e->getMessage();
}

// Cerrar la conexión
$pdo = null;
```

Opciones adicionales

La clase PDO en PHP ofrece una variedad de opciones que se pueden establecer al crear una instancia de la clase PDO. Estas opciones se pasan como un array de opciones al constructor de PDO. Aquí hay algunas opciones comunes que puedes utilizar:

1. PDO::**ATTR_ERRMODE**: Define el modo de manejo de errores. Puedes establecerlo en:
    - PDO::ERRMODE_EXCEPTION: Lanza excepciones PDOException en caso de errores.
    - PDO::ERRMODE_WARNING: Genera advertencias E_WARNING en caso de errores.
    - PDO::ERRMODE_SILENT: No muestra mensajes de error o advertencias. Debes verificar los errores manualmente.
2. PDO::**ATTR_DEFAULT_FETCH_MODE**: Establece el modo de obtención de resultados por defecto. Puedes establecerlo en:
    - PDO::FETCH_ASSOC: Devuelve los resultados como un array asociativo.
    - PDO::FETCH_NUM: Devuelve los resultados como un array indexado numéricamente.
    - PDO::FETCH_BOTH: Devuelve los resultados como un array asociativo y numérico.
3. PDO::**ATTR_EMULATE_PREPARES**: Indica si se deben emular las consultas preparadas. Puedes establecerlo en:
    - true: Las consultas preparadas se emulan.
    - false: Las consultas preparadas se ejecutan nativamente.
4. PDO::**ATTR_PERSISTENT**: Habilita o deshabilita la conexión persistente. Puedes establecerlo en:
    - true: Habilita la conexión persistente.
    - false: Deshabilita la conexión persistente.
5. PDO::**MYSQL_ATTR_INIT_COMMAND**: Establece un comando SQL que se ejecuta al establecer la conexión con una base de datos MySQL.

6. PDO::**ATTR_FETCH_TABLE_NAMES**: Controla si los nombres de las tablas se incluyen en los resultados al utilizar fetchAll().
    - PDO::FETCH_TABLE_NAMES: Devuelve los resultados con los nombres de las tablas incluidos.
    - PDO::FETCH_CLASS | PDO::FETCH_CLASSTYPE: Devuelve los resultados como una instancia de la clase especificada en fetch() o fetchAll().

Estos son solo algunos ejemplos de opciones disponibles en PDO. Puedes consultar la documentación oficial de PHP para obtener una lista completa de opciones y sus descripciones detalladas.